

SANGOLA MAHAVIDYALAYA SANGOLA

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATION

Subject – .NET Core

DATE- 24/09/2025

.....

Practical Assignment-03

1. Write a program to sort a list of integers using `List<T>.Sort()`.
2. Write a program to reverse a list of strings.
3. Write a program to merge two lists and remove duplicates using `HashSet<T>`.
4. Write a program to find the frequency of words in a sentence using `Dictionary<string,int>`.
5. Write a program to implement a simple student record system using `List<Student>`.
6. Write a program to search for an element in a list using `Contains()`.
7. Write a program to remove duplicates from a list using LINQ `Distinct()`.
8. Write a program to demonstrate stack operations (push, pop, peek).

9. Write a program to demonstrate queue operations (enqueue, dequeue, peek).
10. Write a program to simulate undo/redo functionality using `Stack<T>`.
11. Write a program to simulate a printer queue using `Queue<T>`.
12. Write a program to store key-value pairs using `Dictionary<TKey,TValue>`.
13. Write a program to iterate over a dictionary and print keys and values.
14. Write a program to sort a dictionary by keys.
15. Write a program to sort a dictionary by values.
16. Write a program to check subset and superset relationships using `HashSet<T>`.
17. Write a program to find the intersection of two sets using `HashSet<T>.IntersectWith()`.
18. Write a program to find the union of two sets using `HashSet<T>.UnionWith()`.

19. Write a program to find the difference between two sets using `HashSet<T>.ExceptWith()`.
20. Write a program to implement linked list operations (add, remove, traverse).
21. Write a program to find duplicates in an array using `Dictionary<int,int>`.
22. Write a program to group employees by department using `Dictionary<string,List<Employee>>`.
23. Write a program to count vowels in a string using `Dictionary<char,int>`.
24. Write a program to demonstrate `SortedList<TKey,TValue>`.
25. Write a program to implement a phonebook using `Dictionary<string,string>`.

LINQ With Database

26.Create Employees + Departments + EmployeeProjects tables and solve questions

```
CREATE TABLE Departments (
```

```
    DepartmentId INT PRIMARY KEY IDENTITY,
```

```
    DepartmentName NVARCHAR(100) NOT NULL
```

```
);
```

```
CREATE TABLE Employees (
```

```
    EmployeeId INT PRIMARY KEY IDENTITY,
```

```
    Name NVARCHAR(100) NOT NULL,
```

```
    DepartmentId INT NULL,
```

```
    ManagerId INT NULL,
```

```
    Salary DECIMAL(18,2) NOT NULL,
```

```
    HireDate DATE NOT NULL,
```

```
    FOREIGN KEY (DepartmentId) REFERENCES Departments(DepartmentId),
```

```
    FOREIGN KEY (ManagerId) REFERENCES Employees(EmployeeId)
```

```
);
```

```
CREATE TABLE EmployeeProjects (
```

```
    EmployeeId INT NOT NULL,
```

```
    ProjectId INT NOT NULL,
```

```
    PRIMARY KEY (EmployeeId, ProjectId)
```

);

1. Join Employees with Departments and display employee name with department.
2. Find employees earning more than 50,000 in each department.
3. Count employees per department.
4. Find maximum salary in each department.
5. Find employees without a manager.
6. Group employees by department and order by salary.
7. Find employees hired after 2020.
8. Find employees whose names start with 'A'.
9. Find employees earning above department average.
10. List employees working on projects.

27. Create Students + Courses + StudentCourses Table

CREATE TABLE Students (

 StudentId INT PRIMARY KEY IDENTITY,

 Name NVARCHAR(100) NOT NULL,

 Marks INT NOT NULL,

 Year INT NOT NULL

);

CREATE TABLE Courses (

 CourseId INT PRIMARY KEY IDENTITY,

 CourseName NVARCHAR(100) NOT NULL,

 Credits INT NOT NULL

);

CREATE TABLE StudentCourses (

```

StudentId INT NOT NULL,

CourseId INT NOT NULL,

PRIMARY KEY (StudentId, CourseId)

);

1. Join students with their courses.
2. Find students with marks greater than 80.
3. Find courses taken by each student.
4. Count number of courses per student.
5. Find most popular course.
6. Find students enrolled in more than one course.
7. Sort students by marks.
8. Find average marks of students.
9. Check if all students passed (marks > 40).
10. Skip first 5 students and take next 5.

```

28.Create: Customers + Orders + OrderDetails + Products tables and solve questions

```

CREATE TABLE Customers (

    CustomerId INT PRIMARY KEY IDENTITY,

    Name NVARCHAR(100) NOT NULL

);

CREATE TABLE Orders (

    OrderId INT PRIMARY KEY IDENTITY,

    CustomerId INT NOT NULL,

    OrderDate DATE NOT NULL,

```

TotalAmount DECIMAL(18,2) NOT NULL

);

CREATE TABLE Products (

ProductId INT PRIMARY KEY IDENTITY,

ProductName NVARCHAR(100) NOT NULL,

Price DECIMAL(18,2) NOT NULL

);

CREATE TABLE OrderDetails (

OrderId INT NOT NULL,

ProductId INT NOT NULL,

Quantity INT NOT NULL,

PRIMARY KEY (OrderId, ProductId)

);

1. Join customers with their orders.
2. Find total orders per customer.
3. Calculate total purchase amount per customer.
4. Join orders with products.
5. Find most ordered product.
6. Find total quantity sold per product.
7. Find highest value order.
8. Find customers who placed orders after 2023.
9. Implement paging using Skip & Take.
10. Sort products by price descending.

29. Create Employees + Projects + Departments table and solve questions

```
CREATE TABLE Projects (  
    ProjectId INT PRIMARY KEY IDENTITY,  
    ProjectName NVARCHAR(100) NOT NULL,  
    DepartmentId INT NULL  
);
```

(Used with Employees and Departments from Set 1)

1. Join Employees, Projects, and Departments.
2. List projects handled by each department.
3. Count projects per department.
4. Find employees working on multiple projects.
5. Group employees by project.
6. Find employees in IT department projects.
7. Sort employees by project name.
8. Find departments handling most projects.